

Cloud Computing Option 2: Serverless Image Processing Application on AWS Lambda

Luis Sanchez Juan Garcia Edward King

June 2026

Sapienza University of Rome Cloud Computing

Abstract

[EDDIE: Update abstract after completing the report]

This report presents the design, implementation, deployment, and performance evaluation of a serverless image processing application developed using Amazon Web Services (AWS). The application allows users to upload images through a REST API and perform image processing operations using AWS Lambda functions. The deployment uses API Gateway, Lambda, Amazon S3, and CloudWatch services.

The main objective of the project is to evaluate the scalability and performance of serverless computing under different workload conditions. Experimental tests were conducted using varying numbers of concurrent users and requests. Metrics including response time, throughput, invocation count, execution duration, concurrency level, and error rate were collected and analyzed.

The obtained results demonstrate the ability of AWS Lambda to automatically scale according to workload intensity while maintaining acceptable performance levels. A cost analysis is also presented and compared with alternative deployment solutions.

Contents

1	Introduction	3
1.1	Project Objectives	3
1.2	Project Scope	3
1.3	Purpose and Scope of the Evaluation	4
2	System Design	4
2.1	Overall Architecture	4
2.2	AWS Components	5
2.2.1	API Gateway	5
2.2.2	AWS Lambda	5
2.2.3	Amazon S3	6
2.2.4	Amazon CloudWatch	6
2.3	Features Under Evaluation	6
3	Implementation	7
3.1	Upload Service	7
3.2	Processing Service	7
3.3	Deployment	7
4	Experimental Methodology	8
4.1	Evaluation Objectives	8
4.2	Workload Design	8
4.3	Workload Profile	8
4.4	Burst Workload Evaluation	9
4.5	Workload Scenarios	10
4.6	Testing Tools	10
4.7	Metrics Collected	10
4.8	Experimental Procedure	11
5	Experimental Results	11
5.1	Response Time Analysis	11
5.2	Throughput Analysis	12
5.3	Concurrency Analysis	13
5.4	Scalability Evaluation	13
5.5	Bottleneck Analysis	13
5.6	Error Analysis	14
5.7	Availability Analysis	14
5.8	Workload Verification	15
6	Experimental Limitations	15
7	Cost Evaluation	15
7.1	AWS Lambda Cost Estimation	15
7.2	Alternative Deployment Comparison	15
7.3	Discussion	16
8	Discussion	16
9	Conclusion	16

1 Introduction

Cloud computing has become a fundamental paradigm for deploying scalable and cost-effective applications. Among the various cloud service models, serverless computing has gained significant attention due to its ability to automatically allocate resources according to workload demands while eliminating the need for infrastructure management. Amazon Web Services (AWS) Lambda is one of the most widely adopted serverless platforms, enabling developers to execute application logic without provisioning or maintaining servers.

[EDDIE: Probably have to add some references here] Image processing applications represent a suitable use case for serverless architectures because workloads can fluctuate considerably over time. Traditional server-based deployments often require resources to be provisioned for peak demand, resulting in underutilised infrastructure during periods of low activity. In contrast, serverless deployments dynamically allocate resources according to demand, potentially reducing operational costs while maintaining acceptable performance levels.

] [LUIS: I made some changes to the text below, can you check that it's correct?] This project investigates the deployment of a serverless image processing application using AWS cloud services. The application enables users to upload images through an HTTP API, store them in Amazon S3, and perform image resizing operations using AWS Lambda functions. The deployment integrates Amazon API Gateway, AWS Lambda, Amazon S3, and Amazon CloudWatch to provide a scalable and observable cloud-native architecture.

[LUIS: ALso check the text below, I made some changes to it] The primary focus of the project is not the image processing functionality itself, but rather the evaluation of the performance and scalability characteristics of the serverless deployment. Experimental testing was conducted under varying workload conditions to determine how effectively the system scales in response to increasing demand and to identify potential performance limitations or bottlenecks.

1.1 Project Objectives

The objectives of this project are:

- To design and implement a serverless image processing application using AWS services.
- To deploy the application using AWS Lambda and API Gateway.
- To evaluate the performance of the deployment under different workload conditions.
- To analyse the scalability characteristics of serverless computing.
- To investigate the relationship between workload intensity and resource utilisation.
- To estimate the operational cost of the deployment and compare it with an alternative cloud deployment model.

1.2 Project Scope

[LUIS: I made some changes to the text below, can you check that it's correct?] The developed application provides a simple image processing workflow consisting of image upload, storage, and resizing operations. Users interact with the system through HTTP endpoints exposed by Amazon API Gateway. Uploaded images are stored in Amazon S3, while image processing tasks are executed by AWS Lambda functions.

The scope of the project includes:

[LUIS: Here also changes]

- Image upload through HTTP API endpoints.

- Image storage using Amazon S3.
- Image resizing operations executed by AWS Lambda.
- Monitoring and metric collection through Amazon CloudWatch.
- Experimental performance and scalability evaluation.

The project does not focus on advanced image analysis, machine learning functionality, or front-end application development. Instead, the emphasis is placed on evaluating the behaviour of the serverless infrastructure under controlled workload conditions.

1.3 Purpose and Scope of the Evaluation

The purpose of the performance evaluation is to assess the scalability, responsiveness, and reliability of the proposed serverless image processing application under different workload conditions.

The study aims to determine whether the AWS Lambda-based deployment is capable of automatically scaling in response to increasing demand while maintaining an acceptable level of service. In particular, the evaluation investigates how workload intensity affects response time, throughput, concurrency levels, and overall system behaviour.

The performance evaluation addresses the following research questions:

[EDDIE: Check that these questions have been answered in the report]

- Does the application scale effectively as workload intensity increases?
- How does response time vary under different levels of concurrency?
- What throughput can be achieved under increasing load?
- How many concurrent Lambda executions are required to maintain performance?
- Are there any observable bottlenecks within the processing workflow?
- Does the deployment maintain reliable operation under sustained and burst workloads?

[LUISJUAN: Do we have all of the metrics below?] The evaluation considers both user-oriented and system-oriented performance metrics. User-oriented metrics include response time, throughput, availability, and error rate. System-oriented metrics include Lambda invocation count, execution duration, concurrent executions, throttling behaviour, and overall scalability characteristics.

Experimental data were collected using Amazon CloudWatch and the selected load generation tools. The resulting measurements provide quantitative evidence regarding the performance and scalability of the serverless deployment and support the conclusions presented later in this report.

2 System Design

2.1 Overall Architecture

The proposed solution follows a serverless architecture built entirely using managed AWS services. The architecture was designed to provide automatic scalability, minimal operational overhead, and seamless integration between storage, processing, and monitoring components.

Figure ?? illustrates the overall system architecture.

[LOUIS OR JUAN: The system follows a serverless architecture based on Amazon API Gateway, AWS Lambda, Amazon S3, and Amazon CloudWatch. Users interact with the application through HTTP endpoints exposed by Amazon API Gateway. When a processing request is received, API Gateway invokes an AWS Lambda function that retrieves the input image from an Amazon S3 input bucket, performs the requested image transformation, and stores the processed image in an Amazon S3 output bucket. Amazon CloudWatch is used to collect logs and monitoring metrics such as invocations, duration, concurrency, errors, and throttles. This architecture enables automatic scaling, low operational overhead, and easy monitoring under different workload conditions.

The architecture also includes an optional upload workflow. Through the `/upload-base64` endpoint, users can send an image encoded in Base64 to API Gateway, which triggers a Lambda function that stores the image in the S3 input bucket. Afterwards, the `/process` endpoint triggers the image-processing Lambda function, which reads the uploaded image from the input bucket, applies the requested transformation, and stores the result in the output bucket.

Short flow version for your teammate's diagram Client sends HTTP request to Amazon API Gateway API Gateway invokes AWS Lambda Lambda reads image from Amazon S3 input bucket Lambda processes image Lambda stores result in Amazon S3 output bucket Amazon CloudWatch collects logs and metrics If your teammate wants boxes and arrows

You can tell him this:

Client → API Gateway → Lambda → S3 Input / S3 Output → CloudWatch

More precisely:

Client → API Gateway API Gateway → Lambda Lambda → S3 Input Bucket (read image) Lambda → S3 Output Bucket (write processed image) Lambda / API Gateway → CloudWatch (logs and metrics)]

The image processing workflow consists of the following steps:

1. A client submits an image processing request through a REST API endpoint.
2. AWS API Gateway receives the request and performs request routing.
3. The request is forwarded to an AWS Lambda function responsible for image processing.
4. The Lambda function retrieves or receives the image data and performs the required resizing operation.
5. Input and processed images are stored within Amazon S3.
6. The processed image or processing result is returned to the client through API Gateway.
7. Amazon CloudWatch continuously records operational and performance metrics generated during execution.

[EDDIE: Once that flow diagram is made make sure the paragraph below makes sense] This architecture eliminates the need to provision or manage virtual machines while allowing resources to scale automatically in response to workload fluctuations.

2.2 AWS Components

The deployment relies on four primary AWS services: API Gateway, AWS Lambda, Amazon S3, and Amazon CloudWatch. Together, these services provide the functionality required to implement, operate, and evaluate the serverless image processing application.

2.2.1 API Gateway

Amazon API Gateway provides the external interface through which clients interact with the application. It exposes REST endpoints that allow users to upload images and trigger image processing operations.

API Gateway acts as the entry point of the system, receiving incoming HTTP requests and forwarding them to the appropriate AWS Lambda functions. The service also simplifies request routing, endpoint management, and integration with backend services.

[LOUIS: Insert API endpoint summary table if available.]

2.2.2 AWS Lambda

AWS Lambda provides the serverless execution environment responsible for performing image processing operations.

When a request is received from API Gateway, the corresponding Lambda function is invoked automatically. The function performs the required image resizing operation and stores the resulting image within Amazon S3.

One of the primary advantages of AWS Lambda is its ability to automatically allocate execution resources according to workload demand. This behaviour makes Lambda particularly suitable for workloads characterised by unpredictable traffic patterns and varying request volumes.

The Lambda configuration used during experimentation is summarised in Table 1.

[LOUIS: Confirm Lambda runtime, memory allocation, timeout configuration, and region.]

2.2.3 Amazon S3

Amazon Simple Storage Service (S3) is used as the persistent storage layer for the application.

Uploaded images are stored within S3 buckets before processing, while processed images are also retained for retrieval and verification purposes. The service provides highly durable object storage and integrates directly with AWS Lambda, enabling efficient transfer of image data between storage and processing components.

The use of S3 separates storage responsibilities from computation, allowing the application to scale independently across both dimensions.

[LOUIS: Confirm bucket structure and naming convention used during implementation.]

2.2.4 Amazon CloudWatch

Amazon CloudWatch is used to monitor system behaviour and collect performance metrics throughout experimental testing.

CloudWatch provides visibility into both operational and performance-related aspects of the deployment. Metrics collected during experimentation include invocation count, execution duration, concurrent executions, error count, and throttling events.

These measurements form the basis of the performance evaluation presented later in this report and enable the analysis of system scalability under varying workload conditions.

[JUAN: Insert screenshots or metric dashboard references if required.]

2.3 Features Under Evaluation

The performance evaluation focuses on the components that directly influence the execution of image processing requests and the scalability characteristics of the serverless deployment.

The primary components under evaluation are:

- AWS Lambda image processing functions.

- AWS API Gateway request handling.
- Amazon S3 storage interactions.

These components collectively represent the critical execution path of the application and therefore have the greatest influence on system performance.

The study focuses specifically on the behaviour of the serverless processing pipeline under varying workload conditions. Particular attention is given to response time, throughput, concurrency levels, execution duration, and scaling behaviour.

Several components are intentionally excluded from the evaluation:

- User authentication and authorisation services.
- Front-end interfaces and client-side rendering.
- External network conditions beyond the AWS environment.
- Advanced image processing algorithms unrelated to system scalability.

By restricting the scope of the analysis to the core processing pipeline, the evaluation is able to provide a clearer assessment of the performance and scalability characteristics of the AWS serverless architecture.

The resulting experimental study therefore focuses on determining whether AWS Lambda can efficiently scale image processing workloads while maintaining acceptable levels of responsiveness, throughput, and reliability.

3 Implementation

3.1 Upload Service

The upload endpoint receives an image encoded in Base64 format and stores it in Amazon S3.

Endpoint

/upload-base64

Example Request

```
{  
  "filename": "test.jpg",  
  "folder": "small",  
  "content_type": "image/jpeg",  
  "image_base64": "..."  
}
```

3.2 Processing Service

The processing endpoint performs image resizing operations.

Endpoint

/process

Example Request

```
{
  "bucket": "bucket-name",
  "key": "image.jpg",
  "operation": "resize",
  "width": 256,
  "height": 256
}
```

3.3 Deployment

The application was deployed within the AWS environment using:

- API Gateway
- AWS Lambda
- Amazon S3
- CloudWatch

Deployment configuration parameters are summarized in Table 1.

Table 1: Deployment Configuration

Parameter	Value
Lambda Runtime	TBD
Memory Allocation	TBD
Timeout	TBD
Region	TBD

4 Experimental Methodology

4.1 Evaluation Objectives

The evaluation focuses on the following metrics:

- Response Time
- Throughput
- Scalability
- Concurrent Executions
- Error Rate

4.2 Workload Design

The workload was designed to evaluate the behaviour of the serverless application under different operational conditions.

Requests consisted of image resizing operations performed through the REST API exposed by API Gateway.

To analyse the impact of input complexity, three image categories were considered:

- Small images
- Medium images
- Large images

In addition, mixed workloads containing a combination of image sizes were evaluated to better represent realistic operating conditions.

4.3 Workload Profile

Each experiment followed a workload profile designed to observe both scale-out and scale-in behaviour.

The workload consisted of five phases:

1. Warm-up (WU)
2. Ramp-up (RU)
3. Steady-State (SS)
4. Ramp-down (RD)
5. Cool-down (CD)

During the warm-up phase, a small number of requests were generated to initialise the Lambda execution environment and reduce cold-start effects.

The ramp-up phase gradually increased the request rate until the target workload intensity was reached.

The steady-state phase maintained a constant workload to observe stable system behaviour.

The ramp-down phase gradually reduced the workload to evaluate the ability of the deployment to scale in.

Finally, the cool-down phase allowed the system to return to idle conditions.

Figure 1 illustrates the workload profile used during testing.



Figure 1: Workload profile used during performance testing.

4.4 Burst Workload Evaluation

In addition to the continuous workload profile, burst workload tests were performed to evaluate the ability of the deployment to respond to sudden increases in demand.

The burst workload consisted of a rapid increase in concurrent requests, followed by a return to normal operating conditions.

This experiment was designed to evaluate:

- Lambda scaling responsiveness.
- Temporary response time degradation.
- Concurrency growth behaviour.
- Recovery time after workload reduction.

4.5 Workload Scenarios

Four workload levels were considered.

Table 2: Workload Scenarios

Scenario	Concurrent Users
Small	10
Medium	50
Heavy	100
Stress	Until saturation

4.6 Testing Tools

The following tools were used:

- Apache JMeter
- AWS CloudWatch
- Postman

4.7 Metrics Collected

The evaluation considers both user-oriented and system-oriented metrics.

User-Oriented Metrics

- Average Response Time
- Throughput (requests/second)
- Error Rate
- Availability

System-Oriented Metrics

- Lambda Invocation Count
- Lambda Duration
- Concurrent Executions
- Throttling Events
- Scalability Behaviour

Metrics were collected using AWS CloudWatch and Apache JMeter.

4.8 Experimental Procedure

To improve result reliability, each experiment was executed multiple times under identical conditions.

For each workload level:

- The experiment was repeated five times.
- CloudWatch metrics were collected throughout the execution.
- JMeter recorded client-side performance measurements.

- Average values were computed across all runs.

This approach reduces the impact of transient fluctuations and provides more representative performance measurements.

5 Experimental Results

5.1 Response Time Analysis

Figure 2 shows the relationship between workload intensity and response time.

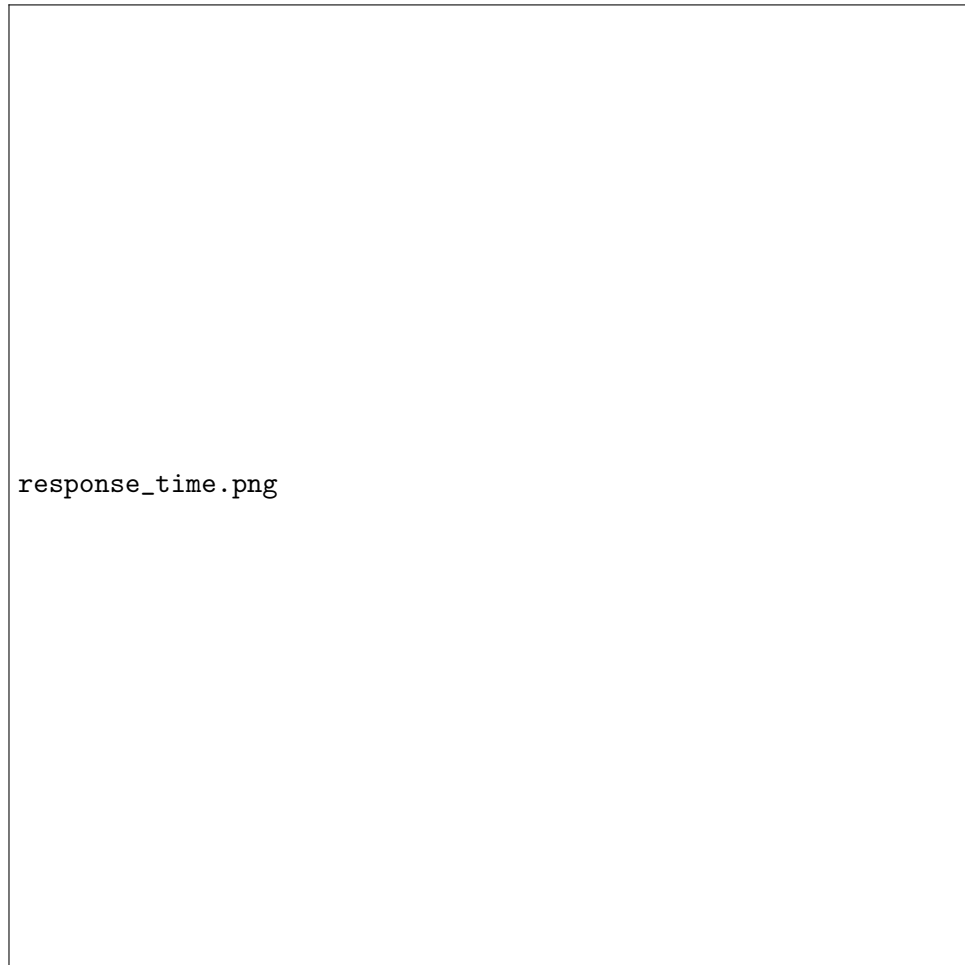


Figure 2: Response Time vs Concurrent Users

Discussion of observed trends goes here.

5.2 Throughput Analysis



Figure 3: Throughput vs Concurrent Users

Discussion of throughput behavior goes here.

5.3 Concurrency Analysis

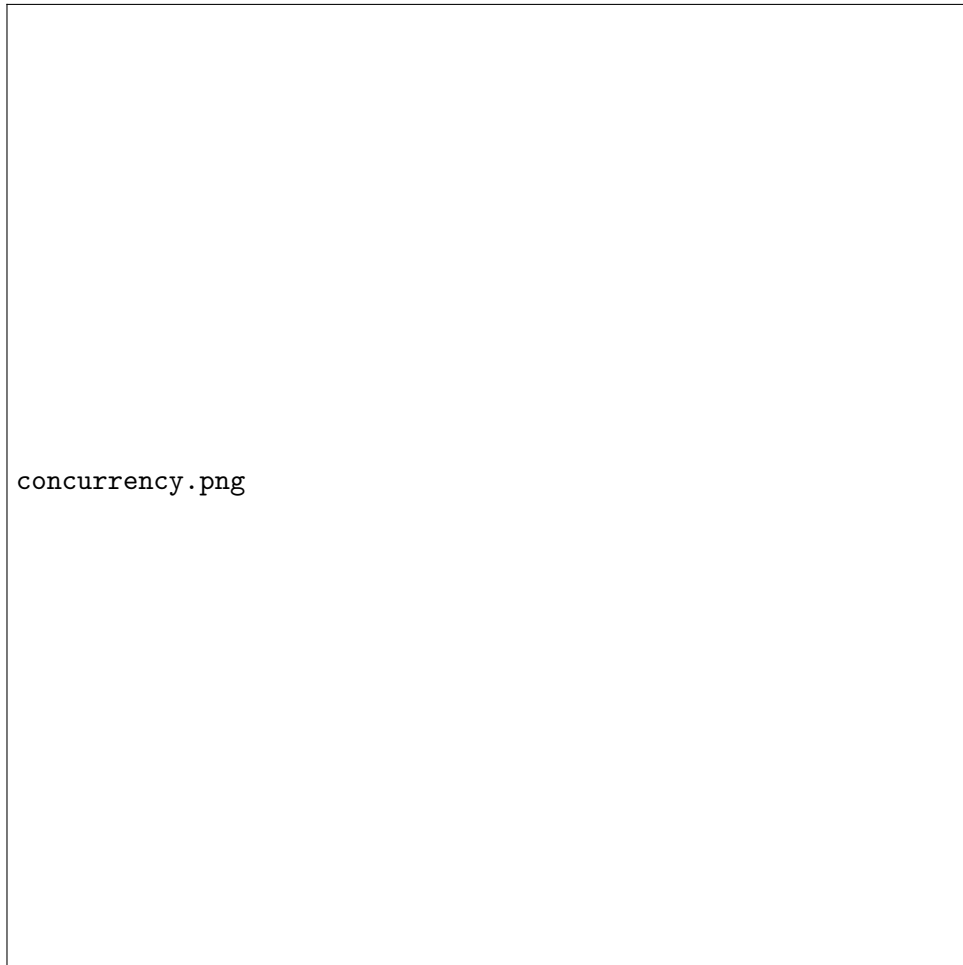


Figure 4: Concurrent Lambda Executions

CloudWatch observations and discussion go here.

5.4 Scalability Evaluation

This section discusses:

- Automatic scaling behavior.
- Maximum observed concurrency.
- Performance bottlenecks.
- Resource utilization.

5.5 Bottleneck Analysis

Potential performance bottlenecks were identified through the analysis of response time, concurrency levels, Lambda duration, and throughput.

The objective is to determine whether performance degradation originates from:

- Lambda execution limits.

- Memory allocation constraints.
- API Gateway limitations.
- Amazon S3 interaction delays.

Observed bottlenecks and their impact on system scalability are discussed using the experimental results.

5.6 Error Analysis

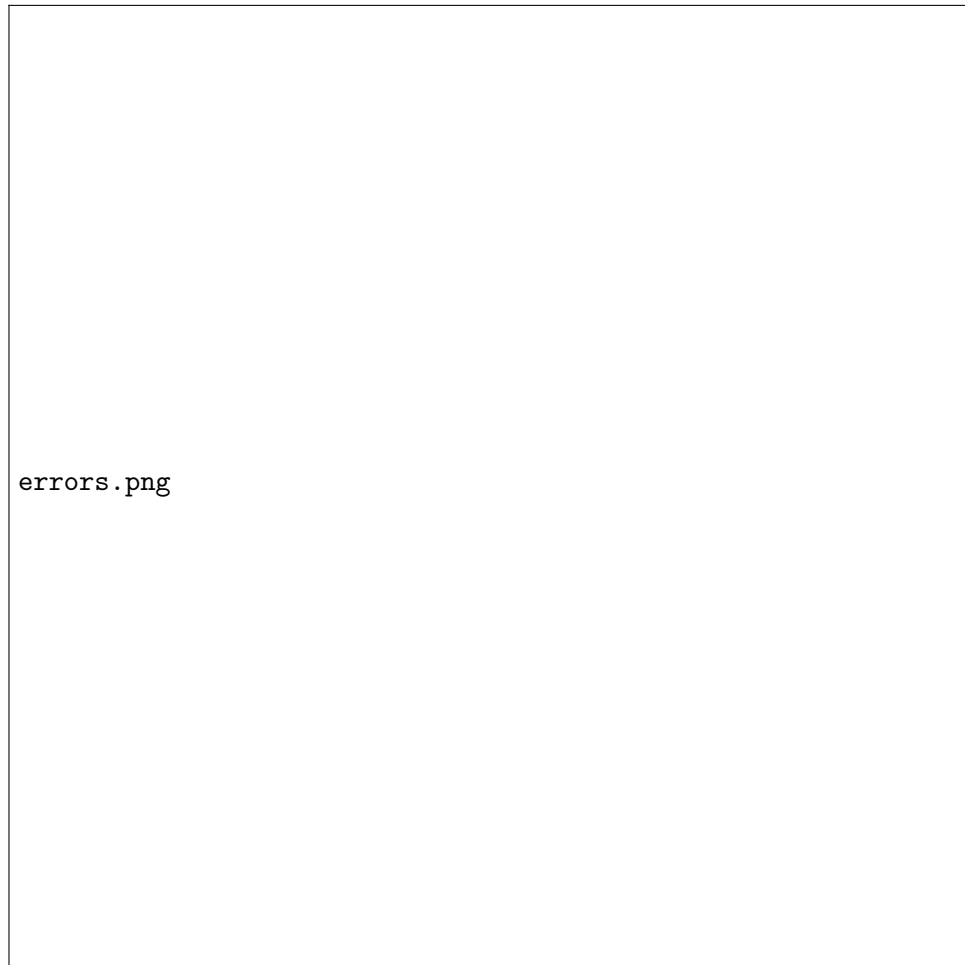


Figure 5: Error Rate vs Concurrent Users

Discussion of system reliability under load.

5.7 Availability Analysis

System availability was evaluated throughout all experimental runs.

Availability was calculated as:

$$Availability = \frac{Successful\ Requests}{Total\ Requests}$$

or equivalently

$$Availability = 1 - ErrorRate$$

The results indicate the ability of the deployment to maintain service continuity during increasing workload conditions.

5.8 Workload Verification

Before analysing performance metrics, the workload generated by JMeter was compared with the workload observed by AWS CloudWatch.

This verification ensures that the request rate generated by the load generator matches the workload effectively received by the system.

The comparison confirms that network limitations or configuration issues did not significantly alter the intended workload profile.

6 Experimental Limitations

Several factors may influence the obtained results:

- AWS Learner Lab resource limitations.
- Variability in cloud resource allocation.
- Internet latency fluctuations.
- Limited workload intensity imposed by AWS usage policies.
- Limited number of repetitions due to budget constraints.

Despite these limitations, the collected results provide a representative view of the scalability and performance characteristics of the deployment.

7 Cost Evaluation

7.1 AWS Lambda Cost Estimation

A six-month operational cost estimate is provided.

Table 3: Estimated Operational Cost

Component	Monthly Cost	6-Month Cost
Lambda	TBD	TBD
API Gateway	TBD	TBD
S3 Storage	TBD	TBD
CloudWatch	TBD	TBD
Total	TBD	TBD

7.2 Alternative Deployment Comparison

The proposed serverless solution is compared with an EC2-based deployment.

Table 4: Deployment Comparison

Metric	Lambda	EC2
Cost	TBD	TBD
Scalability	TBD	TBD
Management Effort	TBD	TBD
Availability	TBD	TBD

7.3 Discussion

Advantages and disadvantages of both approaches are discussed in this section.

8 Discussion

The experimental results indicate:

- Whether the system scales effectively.
- The workload levels at which degradation begins.
- The effectiveness of serverless deployment.
- Observed limitations and bottlenecks.

9 Conclusion

This project demonstrated the deployment of a serverless image processing application using AWS Lambda. Experimental evaluation showed how the system behaves under increasing workload intensity and highlighted the scalability characteristics of serverless computing. The results collected through CloudWatch and JMeter provide quantitative evidence of system performance. Finally, cost analysis demonstrated the economic implications of adopting a serverless architecture compared to traditional deployment solutions.

References

- [1] Amazon Web Services Documentation. <https://docs.aws.amazon.com>
- [2] Apache JMeter Documentation. <https://jmeter.apache.org>
- [3] Cloud Computing Course Material.